

Chapter 04

Programming Principles

Object-Oriented Programming

Inheritance and Polymorphism

Mr. SOK Pongsametrey
metreysk@gmail.com

This week we connect inheritance and polymorphism to Programming Principles such as DRY, modularity, reusability, maintainability, and separation of concerns.

ABOUT

What is Inheritance?

- A mechanism where one class acquires properties and behaviors of another class
- **Parent class** (superclass) provides common features
- **Child class** (subclass) inherits and can add specialized features
- Uses the extends keyword in Java
- **Real-world analogy:** Animal → Dog, Cat; Vehicle → Car, Truck
- **Principle tie-in: Promotes DRY by reusing common code, supports modularity**

Basic Inheritance Example

```
// Parent class (superclass)
class Animal {
    String name;
    int age;

    void eat() {
        System.out.println(name + " is eating");
    }

    void sleep() {
        System.out.println(name + " is sleeping");
    }
}

// Child class (subclass)
class Dog extends Animal {
    String breed;

    void bark() {
        System.out.println(name + " is barking");
    }
}
```

- **Principle tie-in: Common behaviors defined once, specialized behaviors added separately (separation of concerns)**

Using Inherited Classes

```
public static void main(String[] args) {  
    Dog myDog = new Dog();  
    myDog.name = "Buddy";  
    myDog.age = 3;  
    myDog.breed = "Golden Retriever";  
  
    // Inherited methods  
    myDog.eat();    // "Buddy is eating"  
    myDog.sleep();  // "Buddy is sleeping"  
  
    // Dog-specific method  
    myDog.bark();   // "Buddy is barking"  
}
```

- **Principle tie-in: Child objects can use both inherited and their own methods(reusability)**

The `super` Keyword

```
class Animal {
    String name;

    public Animal(String name) {
        this.name = name;
    }

    void makeSound() {
        System.out.println("Some generic animal sound");
    }
}

class Dog extends Animal {
    public Dog(String name) {
        super(name); // Call parent constructor
    }

    void makeSound() {
        super.makeSound(); // Call parent method
        System.out.println("Woof!");
    }
}
```

- `super()` calls parent constructor
- `super.methodName()` calls parent method
- **Principle tie-in: Extends behavior without rewriting everything (DRY)**

What is Polymorphism?

- "Many forms" - one interface, multiple implementations
- Same method name, different behaviors in different classes
- Achieved through method overriding
- Enables flexible, extensible code design
- **Principle tie-in: Supports Open/Closed Principle (open for extension, closed for modification)**

Method Overriding Example

```
class Animal {  
    void makeSound() {  
        System.out.println("Some animal sound");  
    }  
}  
  
class Cat extends Animal {  
    @Override  
    void makeSound() {  
        System.out.println("Meow");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void makeSound() {  
        System.out.println("Woof");  
    }  
}
```

- @Override annotation ensures correct overriding
- **Principle tie-in: Each class handles its own responsibility (Single Responsibility Principle)**

Polymorphism in Action

```
public static void main(String[] args) {  
    Animal[] animals = {  
        new Dog("Buddy"),  
        new Cat("Whiskers"),  
        new Dog("Max")  
    };  
  
    // Polymorphism: same method call, different behaviors  
    for (Animal animal : animals) {  
        animal.makeSound(); // Calls appropriate overridden  
method  
    }  
    // Output:  
    // Woof  
    // Meow  
    // Woof  
}
```

- **Principle tie-in: Write once, work with many types (DRY and maintainability)**

Real-world AI Example – Chatbot Hierarchy

```
abstract class Chatbot {
    String name;

    public Chatbot(String name) {
        this.name = name;
    }

    abstract String generateResponse(String userInput);

    void logConversation(String message) {
        System.out.println "[" + name + "]: " + message;
    }
}

class CustomerSupportBot extends Chatbot {
    public CustomerSupportBot() {
        super("SupportBot");
    }

    @Override
    String generateResponse(String userInput) {
        return "Thank you for contacting support. How can I help you?";
    }
}
```

- **Principle tie-in: Each bot type handles its domain expertise (separation of concerns)**

Abstract Classes and Methods

```
abstract class Shape {
    String color;

    public Shape(String color) {
        this.color = color;
    }

    // Abstract method - must be implemented by subclasses
    abstract double calculateArea();

    // Concrete method - can be inherited as-is
    void displayColor() {
        System.out.println("Color: " + color);
    }
}

class Circle extends Shape {
    double radius;

    public Circle(String color, double radius) {
        super(color);
        this.radius = radius;
    }

    @Override
    double calculateArea() {
        return Math.PI * radius * radius;
    }
}
```

- Principle tie-in: Enforces consistent interface while allowing specialized implementations

SOLID Principles Connection

- Single Responsibility Principle (SRP):
 - Each class has one reason to change Open/Closed Principle (OCP): Open for extension, closed for modification,
 - Liskov Substitution Principle (LSP): Subclasses should be substitutable for their base classes

```
// LSP Example
void makeAnimalsSound(Animal[] animals) {
    for (Animal animal : animals) {
        animal.makeSound(); // Works with any Animal
    }
}
```

Principle tie-in: Inheritance and polymorphism naturally support SOLID design

SOLID Principles Explained: SRP, OCP, and LSP

Single Responsibility Principle (SRP)

"One class, one job"

- Bad: A Student class that handles data, calculations, database, reports, AND emails
- Good: Separate classes - Student (data), GPACalculator (math), StudentRepository (database), etc.
- Benefit: Easier to maintain, test, and modify

SOLID Principles Explained: SRP, OCP, and LSP

Open/Closed Principle (OCP)

"Open for extension, closed for modification"

- Bad: Adding new shapes requires modifying the AreaCalculator class
- Good: Abstract Shape class with calculateArea() method - new shapes extend without changing existing code
- Benefit: Add features safely without breaking existing functionality

SOLID Principles Explained: SRP, OCP, and LSP

Liskov Substitution Principle (LSP)

"Subclasses must work wherever parent class works"

- Bad: Penguin extends Bird but throws exception when fly() is called
- Good: Separate Bird class from Flyable interface - only flying birds implement Flyable
- Benefit: Reliable inheritance that doesn't break existing code

SOLID Principles Explained: SRP, OCP, and LSP



Key Insights:

1. SRP makes classes focused and maintainable
2. OCP makes systems extensible and safe
3. LSP makes inheritance reliable and predictable

Summary

Principle	Key Point	Benefit
SRP	One class, one responsibility	Easier to maintain and test
OCP	Extend behavior, don't modify existing code	Safer to add new features
LSP	Subclasses must work wherever parent works	More reliable inheritance

These principles work together to create **maintainable**, **extensible**, and **reliable** object-oriented code!

When to Use Inheritance

✓ Use inheritance when:

- There's a clear "is-a" relationship (Dog IS-A Animal)
- You want to share common behaviour
- You need polymorphic behavior
- Base class is stable and well-designed

✗ Avoid inheritance when:

- Relationship is "has-a" (use composition instead)
- Parent class is unstable or frequently changing
- Inheritance creates unnecessary complexity
- You only need a few methods from parent

Lab Activity - Shape Hierarchy

Create a Shape System

- Create an abstract Shape class with:
 - Abstract method: calculateArea()
 - Concrete method: displayInfo()
- Create subclasses: Rectangle, Circle, Triangle
- Each subclass implements calculateArea() appropriately
- Create an array of different shapes and calculate total area using polymorphism
- **Focus: Apply inheritance, method overriding, and polymorphism**

Exercise - Employee Management System

Create an Employee Hierarchy

- Abstract Employee class with: name, id, salary
- Abstract method: calculateBonus()
- Concrete method: displayDetails()
- Subclasses: Manager, Developer, Salesperson
- Each has different bonus calculation logic
- Demonstrate polymorphism in main()

Reflection Questions

- How do inheritance and polymorphism improve code maintainability?
- Which SOLID principles did you apply?

Due: Next class

Key Takeaways

Programming Principles:

- **DRY:** Inheritance eliminates code duplication
- **Modularity:** Each class focuses on its specific responsibilities
- **Maintainability:** Changes to parent affect all children consistently
- **Extensibility:** Add new types without modifying existing code

SOLID Support:

- **SRP:** Each class has one responsibility
- **OCP:** Open for extension, closed for modification
- **LSP:** Subclasses are substitutable

Wrap-up and Q&A

- Inheritance promotes code reuse and establishes relationships
- Polymorphism enables flexible, extensible designs
- super allows extending behaviour without replacing it
- Abstract classes enforce consistent interfaces
- These concepts directly support SOLID principles and Programming Principles
- **Next Week: Exception Handling and File I/O**